

PRÁCTICO N°4: CONTROL DE ERROR

PARTE 1

1. Dado un byte de 7 bits, ¿cuántos errores puede detectar con paridad par y cuantos con paridad impar (Cantidad par e impar respectivamente de errores)?

Un bit de paridad (sea par o impar) detecta todos los errores de número impar de bits y no detecta ninguno de número par.

Para un bloque de n bits de datos, la cantidad de errores detectables es:

$$\sum_{\substack{k=1 \\ k \text{ impar}}}^n \binom{n}{k}$$

Y los no detectables son:

$$\sum_{\substack{k=0 \\ k \text{ par}}}^n \binom{n}{k}$$

Errores detectables: $\binom{7}{1} + \binom{7}{3} + \binom{7}{5} + \binom{7}{7} = 64$

Errores NO detectables: $\binom{7}{0} + \binom{7}{2} + \binom{7}{4} + \binom{7}{6} = 64$

Por simetría del binomio:

$$\sum_{\substack{k=1 \\ k \text{ impar}}}^n \binom{n}{k} = \sum_{\substack{k=0 \\ k \text{ par}}}^n \binom{n}{k} = 2^{n-1}$$

por lo tanto:

$$\boxed{\text{Detecta } 2^{n-1} \text{ errores}} \text{ y } \boxed{\text{NO Detecta } 2^{n-1} \text{ errores}}$$

$$2^{7-1} = 2^6 = 64$$

- Detecta 64 patrones de error
- No detecta 64 patrones de error

Y vale igual para paridad par o impar.

2. Calcular la redundancia de la codificación dos entre cinco y tres entre siete.

CÓDIGO 2 (UNOS) ENTRE 5 BITS						
PESOS	6	3	2	1	0	
1	0	0	0	1	1	
2	0	0	1	0	1	
3	0	1	0	0	1	
4	0	1	0	1	0	
5	0	1	1	0	0	
6	1	0	0	0	1	
7	1	0	0	1	0	
8	1	0	1	0	0	
9	1	1	0	0	0	
0	0	0	1	1	0	

$\begin{pmatrix} 5 \\ 2 \end{pmatrix}$

10 CÓDIGOS VÁLIDOS
SOBRE 32 POSIBILIDADES DISTINTAS

CASO 1: Dos entre cinco

La redundancia se calcula como: $R = \frac{N}{K}$

$$N = 5$$

$$K = \log_2(10) = 3.32$$

Luego:

$$R = \frac{5}{3.32} = 1.51$$

CASO 2: Tres entre siete

$$N = 7$$

$$\text{códigos válidos} = \binom{7}{3} = 35$$

$$K = \log_2(35) = 5.13$$

$$R = \frac{7}{5.13} = 1.36$$

Como en el caso 2 $R = 1.36$ es más cercana a 1, presenta una mejor redundancia.

3. Si $p = 0,001$ y $m = 100$

a. ¿Cuál es la probabilidad de no error?

La probabilidad de no error es:

$$(1 - p)^m = (1 - 0.001)^{100} = 0.905$$

b. ¿Cuál es la probabilidad de n y de hasta n errores?

$$P(n) = \binom{100}{n} * 0.001^n * 0.999^{100-n}$$

Así, obtendríamos de n errores.

Y la probabilidad de hasta n errores es:

$$P(\leq n) = \sum_{k=0}^n \binom{100}{k} * 0.001^k * 0.999^{100-k}$$

4. Calcular el chequeo de paridad por palabras para RACING e INDEPENDIENTE.

A	0	1	0	0	0	0	0	1
B	0	1	0	0	0	0	1	0
B	0	1	0	0	0	0	1	0
C	0	1	0	0	0	0	1	1
SOH	0	0	0	0	0	0	1	0

CADA BIT DE **SOH** ES EL BIT DE PARIDAD PAR DE CADA COLUMNA

DETECTA RÁFAGAS DE HASTA 15 BITS DE LONGITUD

RAFAGAS DE LONGITUD 16 SERÍAN ERRORES DOBLES POR COLUMNAS (INDETECTABLES)

Utilizando tabla ASCII con 7 bits:

R	1	0	1	0	0	1	0
A	1	0	0	0	0	0	1
C	1	0	0	0	0	1	1
I	1	0	0	1	0	0	1
N	1	0	0	1	1	1	0
G	1	0	0	0	1	1	1
SOH	0	0	1	0	0	0	0

I	1	0	0	1	0	0	1
N	1	0	0	1	1	1	0
D	1	0	0	0	1	0	0
E	1	0	0	0	1	0	1
P	1	0	1	0	0	0	0
E	1	0	0	0	1	0	1
N	1	0	0	1	1	1	0
D	1	0	0	0	1	0	0
I	1	0	0	1	0	0	1
E	1	0	0	0	1	0	1
N	1	0	0	1	1	1	0
T	1	0	1	0	1	0	0
E	1	0	0	0	1	0	1
SOH	1	0	0	1	0	1	0

5. ¿Cuáles son las características de la codificación anterior?

El método usa paridad vertical por columnas con SOH como byte de control. Detecta cualquier error de 1 bit y ráfagas de hasta 15 bits, pero no detecta errores dobles en la misma columna ni ráfagas de 16 bits. No corrige, solo detecta.

6. Chequear el siguiente código ISBN(10) 0-13165332-6; utilice Cryptool2. Encontrar el dígito verificador 950-311192-“?”. Hacerlo manualmente con Cryptool2.

0	10	0
1	9	9
3	8	24
1	7	7
6	6	36
5	5	25
3	4	12
3	3	9
2	2	4
6	1	6
		132

Como $132 \bmod 11 = 0$, el código está correcto.

9	10	90
5	9	45
0	8	0
3	7	21
1	6	6
1	5	5
1	4	4
9	3	27
2	2	4
?	1	?
		202

$$202 \bmod 11 = 4$$

$$(4 + 1 * ?) \bmod 11 = 0$$

$$(4 + 1 * 7) \bmod 11 = 0$$

9	10	90
5	9	45
0	8	0
3	7	21
1	6	6
1	5	5
1	4	4
9	3	27
2	2	4
7	1	7
		209

Luego, $209 \bmod 11 = 0$.

7. En un código de base 37 codifique la siguiente secuencia: ABCDE-? 12345-? Compruebe que una permutación en la secuencia se detecta como error.

$$ABCDE-? = 10 * 1 + 11 * 2 + 12 * 3 + 13 * 4 + 14 * 5 + ? * 6 = (190 + ? * 6) \bmod 37 = 0$$

$$ABCDE - U = 10 * 1 + 11 * 2 + 12 * 3 + 13 * 4 + 14 * 5 + 30 * 6 = (370) \bmod 37 = 0$$

$$12345-? = 1 * 1 + 2 * 2 + 3 * 3 + 4 * 4 + 5 * 5 + ? * 6 = (55 + ? * 6) \bmod 37 = 0$$

$$55 + 34 * 6 = 259$$

$$12345 - Y = 1 * 1 + 2 * 2 + 3 * 3 + 4 * 4 + 5 * 5 + 34 * 6 = 259 \bmod 37 = 0$$

Comprobamos que una permutación se detecta:

Tomamos ABCDE-U y permutamos C y D: ABDCE-U.

$$ABDCE - U = 10 * 1 + 11 * 2 + 13 * 3 + 12 * 4 + 14 * 5 + 30 * 6 = (369) \bmod 37 = 36 \neq 0$$

La condición ya no se cumple, por lo que la permutación se detecta como error.

8. A los códigos anteriores; ¿por qué se los denomina pesados? Verifique los códigos anteriores por este procedimiento.

Son *pesados* porque el peso de cada carácter incluye su posición i .

La condición de validez es que la suma de productos “peso posicional $\times$ valor del carácter”, es decir, $\sum i \cdot v_i$ sea congruente con 0 módulo 37.

Ese producto $i \cdot v_i$ es el peso total de cada símbolo en la palabra; por eso el código se denomina “pesado”.

La verificación “por este procedimiento” es justamente lo que hicimos:

Para ABCDE-U y para 12345-Y se calculó $\sum i \cdot v_i$ y se comprobó que el resultado es 0 módulo 37, por lo que ambas palabras son válidas según el código pesado de base 37.

9. Investigue el motivo de la conveniencia para el uso de la base prima en los sistemas de codificación.

Si probamos la ecuación típica de los códigos ponderados:

$$\sum (\text{peso} \times \text{dígito}) \equiv 0 \pmod{B}$$

Si ocurre un error en un solo dígito, el receptor ve:

$$\text{Error} = \Delta \cdot \text{peso}$$

El sistema solo puede detectar el error si esa multiplicación NO es $\equiv 0 \pmod{B}$.

Si dos errores distintos producen el mismo resto módulo B, NO se pueden distinguir $\rightarrow$ mala decodificación.

En una base prima:

- Ningún número distinto de 0 tiene inverso nulo
- No existe $\Delta \neq 0$ tal que $\Delta \cdot k \equiv 0 \pmod{p}$

Esto evita que dos errores distintos produzcan la misma firma $\rightarrow$ se pueden detectar y corregir.

En una base NO prima no se distingue, un valor diferente produce el mismo resultado $\rightarrow$ ¡COLISIÓN! NO hay unicidad en la corrección. Esto hace imposible distinguir errores.

PARTE 2

1. Discutir la redundancia en las distintas variantes para códigos rectangulares con veinticuatro bits de información.

Rectángulo de 4×6 :

*	*	*	*	*	*	*	#
*	*	*	*	*	*	*	#
*	*	*	*	*	*	*	#
*	*	*	*	*	*	*	#
#	#	#	#	#	#	#	

$$R = \frac{N}{K} = \frac{24 + 10}{24} = \frac{34}{24} = 1.42$$

Rectángulo de 3×8 :

*	*	*	*	*	*	*	*	*	#
*	*	*	*	*	*	*	*	*	#
*	*	*	*	*	*	*	*	*	#
#	#	#	#	#	#	#	#	#	

$$R = \frac{N}{K} = \frac{24 + 11}{24} = \frac{35}{24} = 1.46$$

Rectángulo de 2×12 :

*	*	*	*	*	*	*	*	*	*	*	*	*	#
*	*	*	*	*	*	*	*	*	*	*	*	*	#
#	#	#	#	#	#	#	#	#	#	#	#	#	

$$R = \frac{N}{K} = \frac{24 + 14}{24} = \frac{38}{24} = 1.58$$

Como se puede ver, mientras más nos aproximemos a un cuadrado (4×6), menor redundancia tendremos.

2. Estudiar el uso del bit esquina para codificación con paridad par e impar.

El bit esquina es un bit adicional que se coloca en el extremo de una matriz de bits (como si fuera la “esquina inferior derecha”), cuando usamos codificación por paridad en dos dimensiones:

- FILAS → paridad horizontal
- COLUMNAS → paridad vertical
- ESQUINA → paridad de paridades

Se construye una tabla así:

	Col 1	Col 2	Col 3	Paridad columna
Fila 1	b11	b12	b13	P1
Fila 2	b21	b22	b23	P2
Fila 3	b31	b32	b33	P3
Paridad fila	C1	C2	C3	BIT ESQUINA

- Los bits originales son los de la zona verde
- Los bits P (fila) y C (columna) son paridades añadidas
- El bit de la esquina = paridad de TODAS las paridades

El bit de esquina sirve para:

Detección (y en algunos casos corrección) de errores bidimensional

Sin bit esquina, sólo sabemos si hay error, pero NO si afecta filas y columnas simultáneamente.

El sistema crea 3 niveles de paridad:

- 1) Paridad por filas
- 2) Paridad por columnas
- 3) Paridad global del resto de las paridades

Si se produce 1 solo error, afecta una fila y una columna → se puede localizar (corrección 1-bit)

Si se producen 2 errores, rompe las cuentas y el bit esquina lo detecta (pero NO corrige)

Se puede usar paridad par o paridad impar, pero debe usarse la misma política en todas las paridades.

El bit esquina también se calcula según la paridad elegida.

3. Dado un rectángulo de cinco por cuatro codificar lo siguiente: 01001101011001010101

Con un bit de paridad par:

0	1	0	0	1
1	1	0	1	1
0	1	1	0	0
0	1	0	1	0
0	1	0	1	0
1	1	1	1	

Con un bit de paridad impar:

0	1	0	0	0
1	1	0	1	0
0	1	1	0	1
0	1	0	1	1
0	1	0	1	1
0	0	0	0	

4. Deducir las redundancias de la codificación rectangular y triangular.

Redundancia Rectangular:

$$R = \frac{N}{K} = \frac{(L+1)^2 - 1}{L^2} = \frac{L^2 + 2L + 1 - 1}{L^2} = \frac{1+2}{L}$$

Redundancia Triangular:

$$R = \frac{N}{K} = \frac{(P+1)\left(\frac{P}{2}\right) + (P+1)}{(P+1)\left(\frac{P}{2}\right)} = \frac{1+2}{P}$$

5. Codificar según Hamming los siguientes dígitos de información 1010 ¿Cuál es la distancia de un código Hamming? y ¿cuál su capacidad de corrección y/o detección?

El código Hamming se conforma por el conjunto $(n, k, d_{\min})$, donde:

$$n = k + r; \quad d_{\min} = 3; \quad k = 4$$

$$2^r \geq k + r + 1 = n + 1$$

Si $r = 2$:

$$2^2 = 4 \not\geq 4 + 2 + 1 = 7$$

Si $r = 3$:

$$2^3 = 8 \geq 4 + 3 + 1 = 8$$

Luego $r = 3$, por lo que hay 3 chequeadores. Y $n = 7$.

- $C_1 = \text{impares} = I_1 \oplus I_2 \oplus I_4$
- $C_2 = \text{dos y cada dos} = I_1 \oplus I_3 \oplus I_4$
- $C_3 = \text{cuatro y cada cuatro} = I_2 \oplus I_3 \oplus I_4$

C ₁	C ₂	I ₁	C ₃	I ₂	I ₃	I ₄
0	0	0	0	0	0	0
1	1	0	1	0	0	1
0	1	0	1	0	1	0
1	0	0	0	0	1	1
1	0	0	1	1	0	0
0	1	0	0	1	0	1
1	1	0	0	1	1	0
0	0	0	1	1	1	1
1	1	1	0	0	0	0
0	0	1	1	0	0	1
1	0	1	1	0	1	0
0	1	1	0	0	1	1
0	1	1	1	1	0	0
1	0	1	0	1	0	1
0	0	1	0	1	1	0
1	1	1	1	1	1	1

Luego la palabra codificada es: 1011010.

La distancia de un código Hamming es $d_{min} = 3$. La distancia de un código Hamming Extendido es $d_{min} = 4$.

El código Hamming detecta dos errores y corrige uno.

6. Si m es la cantidad de chequeadores. ¿Cuál es la probabilidad de que una secuencia aleatoria de $2^m - 1$ bits sea una palabra código para Hamming?

La probabilidad de que una secuencia aleatoria de 2^m bits sea una palabra válida de Hamming es:

$$P = \frac{1}{2^m}$$

Un código Hamming ($2^m - 1, 2^m - 1 - m$) produce:

- Longitud del código:

$$n = 2^m - 1$$

- Cantidad de palabras código válidas:

$$2^k = 2^{2^m-1-m}$$

- Cantidad total de combinaciones posibles de n bits:

$$2^n = 2^{2^m-1}$$

- La probabilidad de que una palabra binaria al azar sea un código válido es:

$$P = \frac{2^{2^m-1-m}}{2^{2^m-1}} = 2^{-m} = \frac{1}{2^m}$$

7. Dado el siguiente mensaje, codificarlo en forma triangular: 101111001101001.

Con un bit de paridad par:

1	0	1	1	1	0
1	0	0	1	1	
1	0	1	0		
0	0	0			
1	1				
0					

8. Localizar un error en la siguiente palabra Hamming 011100010111110 (bit menos significativo a la izquierda).

$$\text{Si } r = 4 \rightarrow 2^4 = 16 \geq n + 1 = 15 + 1 = 16$$

C_1	C_2	I_1	C_3	I_2	I_3	I_4	C_4	I_5	I_6	I_7	I_8	I_9	I_{10}	I_{11}
0	1	1	1	0	0	0	1	0	1	1	1	1	1	0

Luego, el código de Hamming (15, 11, 3):

- $C_1 = \text{impares} = I_1 \oplus I_2 \oplus I_4 \oplus I_5 \oplus I_7 \oplus I_9 \oplus I_{11}$
- $C_2 = \text{dos y cada dos} = I_1 \oplus I_3 \oplus I_4 \oplus I_6 \oplus I_7 \oplus I_{10} \oplus I_{11}$
- $C_3 = \text{cuatro y cada cuatro} = I_2 \oplus I_3 \oplus I_4 \oplus I_8 \oplus I_9 \oplus I_{10} \oplus I_{11}$
- $C_4 = \text{ocho y cada ocho} = I_5 \oplus I_6 \oplus I_7 \oplus I_8 \oplus I_9 \oplus I_{10} \oplus I_{11}$

Los síndromes para esa palabra son:

- $S_1 = C_1 \oplus I_1 \oplus I_2 \oplus I_4 \oplus I_5 \oplus I_7 \oplus I_9 \oplus I_{11} = 0 \oplus 1 \oplus 0 \oplus 0 \oplus 0 \oplus 1 \oplus 1 \oplus 0 = 1 \rightarrow \text{falla}$
- $S_2 = C_2 \oplus I_1 \oplus I_3 \oplus I_4 \oplus I_6 \oplus I_7 \oplus I_{10} \oplus I_{11} = 1 \oplus 1 \oplus 0 \oplus 0 \oplus 0 \oplus 1 \oplus 1 \oplus 0 = 1 \rightarrow \text{falla}$
- $S_3 = C_3 \oplus I_2 \oplus I_3 \oplus I_4 \oplus I_8 \oplus I_9 \oplus I_{10} \oplus I_{11} = 1 \oplus 0 \oplus 0 \oplus 0 \oplus 0 \oplus 1 \oplus 1 \oplus 0 = 0$
- $S_4 = C_4 \oplus I_5 \oplus I_6 \oplus I_7 \oplus I_8 \oplus I_9 \oplus I_{10} \oplus I_{11} = 1 \oplus 0 \oplus 1 \oplus 1 \oplus 1 \oplus 1 \oplus 1 \oplus 0 = 0$

Los síndromes que NO fallaron (S_3, S_4) utilizan todos los bits de información excepto el I_1 , y los síndromes que fallaron, ambos tienen el bit I_1 , por lo que el error podría ser ese bit.

Si cambiamos de valor el bit, ya no tendría que haber error en la palabra:

- $S_1 = C_1 \oplus I_1 \oplus I_2 \oplus I_4 \oplus I_5 \oplus I_7 \oplus I_9 \oplus I_{11} = 0 \oplus 0 \oplus 0 \oplus 0 \oplus 0 \oplus 0 \oplus 1 \oplus 0 = 0$
- $S_2 = C_2 \oplus I_1 \oplus I_3 \oplus I_4 \oplus I_6 \oplus I_7 \oplus I_{10} \oplus I_{11} = 1 \oplus 0 \oplus 0 \oplus 0 \oplus 0 \oplus 1 \oplus 1 \oplus 0 = 0$
- $S_3 = C_3 \oplus I_2 \oplus I_3 \oplus I_4 \oplus I_8 \oplus I_9 \oplus I_{10} \oplus I_{11} = 1 \oplus 0 \oplus 0 \oplus 0 \oplus 0 \oplus 1 \oplus 1 \oplus 0 = 0$
- $S_4 = C_4 \oplus I_5 \oplus I_6 \oplus I_7 \oplus I_8 \oplus I_9 \oplus I_{10} \oplus I_{11} = 1 \oplus 0 \oplus 1 \oplus 1 \oplus 1 \oplus 1 \oplus 1 \oplus 0 = 0$

Luego, la palabra corregida es: 010100010111110

9. Corregir y decodificar el siguiente Hamming 1001111 (bit menos significativo a la izquierda).

$$r = 3 \rightarrow 2^3 = 8 \geq n + 1 = 7 + 1 = 8$$

C ₁	C ₂	I ₁	C ₃	I ₂	I ₃	I ₄
1	0	0	1	1	1	1

Detectamos el error:

- $S_1 = C_1 \oplus I_1 \oplus I_2 \oplus I_4 = 1 \oplus 0 \oplus 1 \oplus 1 = 1 \rightarrow \text{falla}$
- $S_2 = C_2 \oplus I_1 \oplus I_3 \oplus I_4 = 0 \oplus 0 \oplus 1 \oplus 1 = 0$
- $S_3 = C_3 \oplus I_2 \oplus I_3 \oplus I_4 = 1 \oplus 1 \oplus 1 \oplus 1 = 0$

Como el único que falla es S_1 , entonces podemos deducir que el bit que falla es $C_1 = 1$, por lo que cambiando ese bit ya no debería fallar:

- $S_1 = C_1 \oplus I_1 \oplus I_2 \oplus I_4 = 0 \oplus 0 \oplus 1 \oplus 1 = 0$
- $S_2 = C_2 \oplus I_1 \oplus I_3 \oplus I_4 = 0 \oplus 0 \oplus 1 \oplus 1 = 0$
- $S_3 = C_3 \oplus I_2 \oplus I_3 \oplus I_4 = 1 \oplus 1 \oplus 1 \oplus 1 = 0$

Luego, la palabra corregida es: 0001111

Para decodificar la palabra, eliminamos los chequeadores. Luego la palabra decodificada es:

0111

10. ¿Qué sucede en la codificación de Hamming si se agrega un bit de paridad, varía o no la distancia?

$$U_5 = I_1 + I_2 + I_4$$

$$U_6 = I_1 + I_3 + I_4$$

$$U_7 = I_1 + I_2 + I_3$$

Si, la distancia mínima si varia, y es $d_{min} = 4$. También se agrega un chequeador (bit de paridad), que sería:

$$U_8 = I_1 + I_2 + I_3 + I_4 + U_5 + U_6 + U_7$$

- a. Encontrar las matrices de generación y chequeo, y calcular la distancia mínima.

Si la palabra código tiene la forma:

$$c = [I_1 I_2 I_3 I_4 U_5 U_6 U_7 U_8]$$

Y codifico 4 palabras básicas usando las fórmulas de los U_i : (1000; 0100; 0010; 0001)

I ₁	I ₂	I ₃	I ₄	U ₅	U ₆	U ₇	U ₈
1	0	0	0	1	1	1	0
0	1	0	0	1	0	1	1
0	0	1	0	0	1	1	1
0	0	0	1	1	1	0	1

Con ello armo la matriz generadora G, con tamaño $k \times n = 4 \times 8$

$$G = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 & 1 \end{bmatrix}$$

En G:

- Las primeras 4 columnas forman una identidad I_4 .
- Las ultimas 4 columnas son una matriz P .

O sea:

$$G = [I_4 | P]$$

Con

$$P = \begin{bmatrix} 1 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 \\ 1 & 1 & 0 & 1 \end{bmatrix}$$

Para una matriz generadora en forma sistemática $G = [I_k | P]$, una matriz de chequeo/ortogonal H se construye como:

$$H = [P^T | I_{n-k}]$$

En nuestro caso:

$$H = [P^T | I_{8-4}]$$

Siendo H con tamaño $(n - k) \times n = 4 \times 8$

$$P^T = \begin{pmatrix} 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 \end{pmatrix}$$

Luego la matriz chequeadora H es:

$$H = \begin{pmatrix} 1 & 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \end{pmatrix}$$

- b. Construir un arreglo estándar para el código.

El arreglo estándar

Es una tabla donde:

- **Primera fila:** todas las palabras **código válidas** (las que genera G).
- Cada fila siguiente: un **líder de coset** e (patrón de error de peso mínimo que no haya aparecido antes) y luego todas las palabras $e \oplus c$ donde c recorre la primera fila.

Eso particiona todo F_2^8 (las 256 palabras de longitud 8) en 16 filas de 16 palabras cada una.

Fila 0 (líder = 00000000) → el código C (todas las palabras válidas):

00000000 00011101 00100111 00111010 01001011 01010110 01101100 01110001
10001110 10010011 10101001 10110100 11000101 11011000 11100010 11111111

Fila 1 (líder = 00000001):

00000001 00011100 00100110 00111011 01001010 01010111 01101101 01110000
10001111 10010010 10101000 10110101 11000100 11011001 11100011 11111110

Fila 2 (líder = 00000010):

00000010 00011111 00100101 00111000 01001001 01010100 01101110 01110093
10001100 10010001 10101011 10110110 11000111 11011010 11100000 11111101

Fila 3 (líder = 00000100):

00000100 00011001 00100011 00111110 01001111 01010010 01101000 01110101
10001010 10010111 10101101 10110000 11000001 11011100 11100110 11111011

Fila 4 (líder = 00001000):

00001000 00010101 00101111 00110010 01000011 01011110 01100100 01111001
10000110 10011011 10100001 10111100 11001101 11010000 11101010 11110111

Fila 5 (líder = 00010000):

00010000 00001101 00110177 00101010 01011011 01000110 01111100 01100001
10011110 10000011 10111001 10100100 11010101 11001000 11110010 11101111

Fila 6 (líder = 00100000):

00100000 00111101 00000111 00011010 01101011 01110110 01001100 01010001
10101110 10110011 10001001 10010100 11100101 11111000 11000010 11011111

Fila 7 (líder = 01000000):

01000000 01011101 01100111 01111010 00001011 00010110 00101100 00110001
11001110 11010093 11101001 11110100 10000105 10011000 10100010 10111111

Fila 8 (líder = 10000000):

10000000 10011101 10100111 10111010 11001011 11010110 11101100 11110001
00001110 00010011 00101001 00110100 01000101 01011000 01100010 01111111

Fila 9 (líder = 00000011):

00000011 00011110 00100100 00111001 01001000 01010101 01101111 01110010
10001101 10010000 10101010 10110111 11000110 11011011 11100001 11111100

Fila 10 (líder = 00000101):

00000101 00011000 00100010 00111111 01001110 01010011 01101001 01110100
10001011 10010110 10101100 10110001 11000000 11011101 11100111 11111010

Fila 11 (líder = 00000110):

00000110 00011011 00100001 00111100 01001101 01010000 01101010 01110111
10001000 10010101 10101111 10110010 11000011 11011110 11100100 11111001

Fila 12 (líder = 00001001):

00001001 00010100 00101110 00110011 01000010 01011111 01100101 01111000
10000111 10011010 10100000 10111101 11001100 11010001 11101011 11110110

Fila 13 (líder = 00001010):

00001010 00010111 00101101 00110000 01000001 01011100 01100110 01111011
10000100 10011001 10100011 10111110 11001111 11010010 11101000 11110101

Fila 14 (líder = 00001100):

00001100 00010001 00101011 00110110 01000111 01011010 01100000 01111101
10000010 10011111 10100105 10111000 11001001 11010100 11101110 11110011

Fila 15 (líder = 00010010):

00010010 00001111 00110101 00101000 01011001 01000100 01111110 01100011
10011100 10000001 10111011 10100110 11010111 11001010 11110000 11101101

- c. ¿Cuál es la probabilidad de que se produzca un error y no se detecte?

Para un código lineal cualquiera:

$$P(\text{error no detectado}) = \sum_{\substack{c \in C \\ c \neq 0}} p^{w(c)} (1-p)^{n-w(c)}$$

Donde $w(c)$ es el peso de Hamming de la palabra código c , y n es la longitud (acá $n = 8$).

Las palabras validas en nuestro código (8,4) y sus pesos:

- 1 palabra de peso 0 (la 00000000)
- 14 palabras de peso 4
- 1 palabra de peso 8

Entonces

$$P_{\text{no detectado}} = 14p^4(1-p)^{8-4} + 1 * p^8(1-p)^{8-8}$$

Es decir:

$$P_{\text{no detectado}} = 14^4(1-p)^4 + p^8$$

11. Códigos Reed Solomon.

- a. Completar la siguiente tabla:

Capacidad de corrección k bytes	Cantidad de bits por byte	Longitud de palabra valida (N)	Dimensión del campo de Galois (Q)	Grado del polinomio generado R del campo de Galois	Bytes de información (K)	Bytes de chequeo	Capacidad de corrección en ráfaga si entrelazo 75 códigos
20	10	1023	1024	40	983	40	1500
40	8	255	256	80	175	80	3000
12	6	63	64	24	39	24	900
4	8	209	256	8	201	8	300

12. Sea $g(x) = 1 + x + x^2 + x^4 + x^5 + x^8 + x^{10}$ polinomio generador de un código cíclico (15,5).

- a. Encontrar el polinomio de paridad $h(x)$ de este código

$$h(x) = (x^{15} + 1) / g(x)$$

División de polinomios en GF(2)

$$\text{Dividimos: } (x^{15} + 1) \div (1 + x + x^2 + x^4 + x^5 + x^8 + x^{10})$$

$$x^5 + x^3 + x + 1$$

$$x^{10} + x^8 + x^5 + x^4 + x^2 + x + 1 \mid x^{15} + 0x^{14} + \dots + 0x^1 + 1$$

$$\text{Paso 1: } x^{15} \div x^{10} = x^5$$

$$x^5 \cdot (x^{10} + x^8 + x^5 + x^4 + x^2 + x + 1) = x^{15} + x^{13} + x^{10} + x^9 + x^7 + x^6 + x^5$$

$$x^{15} + 1$$

$$- (x^{15} + x^{13} + x^{10} + x^9 + x^7 + x^6 + x^5)$$

$$\overline{x^{13} + x^{10} + x^9 + x^7 + x^6 + x^5 + 1}$$

Paso 2: $x^{13} \div x^{10} = x^3$

$$x^3 \cdot (x^{10} + x^8 + x^5 + x^4 + x^2 + x + 1) = x^{13} + x^{11} + x^8 + x^7 + x^5 + x^4 + x^3$$

$$\begin{array}{r} x^{13} + x^{10} + x^9 + x^7 + x^6 + x^5 + 1 \\ - (x^{13} + x^{11} + x^8 + x^7 + x^5 + x^4 + x^3) \end{array}$$

$$\overline{x^{11} + x^{10} + x^9 + x^8 + x^6 + x^4 + x^3 + 1}$$

Paso 3: $x^{11} \div x^{10} = x$

$$x \cdot (x^{10} + x^8 + x^5 + x^4 + x^2 + x + 1) = x^{11} + x^9 + x^6 + x^5 + x^3 + x^2 + x$$

$$\begin{array}{r} x^{11} + x^{10} + x^9 + x^8 + x^6 + x^4 + x^3 + 1 \\ - (x^{11} + x^9 + x^6 + x^5 + x^3 + x^2 + x) \end{array}$$

$$\overline{x^{10} + x^8 + x^5 + x^4 + x^2 + x + 1}$$

Paso 4: $x^{10} \div x^{10} = 1$

$$1 \cdot (x^{10} + x^8 + x^5 + x^4 + x^2 + x + 1) = x^{10} + x^8 + x^5 + x^4 + x^2 + x + 1$$

$$\begin{array}{r} x^{10} + x^8 + x^5 + x^4 + x^2 + x + 1 \\ - (x^{10} + x^8 + x^5 + x^4 + x^2 + x + 1) \end{array}$$

$$\overline{0}$$

Resultado: Cociente = $x^5 + x^3 + x + 1$, Resto = 0

- b. Encontrar el polinomio código para el mensaje: $m(x) = 1 + x + x^4$; en forma sistemática.

Paso 1: Multiplicar $m(x)$ por $x^{n-k} = x^{10}$

$$m(x) \cdot x^{10} = (1 + x + x^4) \cdot x^{10} = x^{10} + x^{11} + x^{14}$$

Paso 2: Dividir $m(x)x^{10}$ por $g(x)$ para obtener el resto $r(x)$

Dividimos: $(x^{14} + x^{11} + x^{10}) \div (1 + x + x^2 + x^4 + x^5 + x^8 + x^{10})$

$$\overline{x^{10} + x^8 + x^5 + x^4 + x^2 + x + 1 \mid x^{14} + x^{11} + x^{10}}$$

Paso 1: $x^{14} \div x^{10} = x^4$

$$x^4 \cdot (x^{10} + x^8 + x^5 + x^4 + x^2 + x + 1) = x^{14} + x^{12} + x^9 + x^8 + x^6 + x^5 + x^4$$

$$\begin{array}{r} x^{14} + x^{11} + x^{10} \\ - (x^{14} + x^{12} + x^9 + x^8 + x^6 + x^5 + x^4) \end{array}$$

$$\overline{x^{12} + x^{11} + x^{10} + x^9 + x^8 + x^6 + x^5 + x^4}$$

Paso 2: $x^{12} \div x^{10} = x^2$

$$x^2 \cdot (x^{10} + x^8 + x^5 + x^4 + x^2 + x + 1) = x^{12} + x^{10} + x^7 + x^6 + x^4 + x^3 + x^2$$

$$\begin{array}{r} x^{12} + x^{11} + x^{10} + x^9 + x^8 + x^6 + x^5 + x^4 \\ - (x^{12} + x^{10} + x^7 + x^6 + x^4 + x^3 + x^2) \end{array}$$

$$\overline{x^{11} + x^9 + x^8 + x^7 + x^5 + x^3 + x^2}$$

Paso 3: $x^{11} \div x^{10} = x$

$$x \cdot (x^{10} + x^8 + x^5 + x^4 + x^2 + x + 1) = x^{11} + x^9 + x^6 + x^5 + x^3 + x^2 + x$$

$$x^{11} + x^9 + x^8 + x^7 + x^5 + x^3 + x^2$$

$$- (x^{11} + x^9 + x^6 + x^5 + x^3 + x^2 + x)$$

$$\overline{x^8 + x^7 + x^6 + x}$$

$$\text{Resto: } r(x) = x + x^6 + x^7 + x^8$$

Paso 3: Construir el polinomio código

$$\text{En GF}(2): v(x) = m(x)x^{10} + r(x)$$

$$v(x) = (x^{14} + x^{11} + x^{10}) + (x^8 + x^7 + x^6 + x)$$

$$v(x) = x + x^6 + x^7 + x^8 + x^{10} + x^{11} + x^{14}$$

$$\text{Vector: } v = (0, 1, 0, 0, 0, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1)$$

c. Encontrar las matrices generadora y de chequeo.

Paso 1: Matriz generadora G (forma cíclica)

Cada fila i es $x^i \cdot g(x)$ para $i = 0, 1, 2, 3, 4$:

$$\text{Fila 0: } g(x) = 1 + x + x^2 + x^4 + x^5 + x^8 + x^{10}$$

$$\rightarrow \text{Vector: } (1, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0)$$

$$\text{Fila 1: } x \cdot g(x) = x + x^2 + x^3 + x^5 + x^6 + x^9 + x^{11}$$

$$\rightarrow \text{Vector: } (0, 1, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1, 0, 0, 0)$$

$$\text{Fila 2: } x^2 \cdot g(x) = x^2 + x^3 + x^4 + x^6 + x^7 + x^{10} + x^{12}$$

$$\rightarrow \text{Vector: } (0, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1, 0, 0)$$

$$\text{Fila 3: } x^3 \cdot g(x) = x^3 + x^4 + x^5 + x^7 + x^8 + x^{11} + x^{13}$$

$$\rightarrow \text{Vector: } (0, 0, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1, 0)$$

$$\text{Fila 4: } x^4 \cdot g(x) = x^4 + x^5 + x^6 + x^8 + x^9 + x^{12} + x^{14}$$

$$\rightarrow \text{Vector: } (0, 0, 0, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1)$$

Matriz G (cíclica):

1	1	1	0	1	1	0	0	1	0	1	0	0	0	0
0	1	1	1	0	1	1	0	0	1	0	1	0	0	0
0	0	1	1	1	0	1	1	0	0	1	0	1	0	0
0	0	0	1	1	1	0	1	1	0	0	1	0	1	0
0	0	0	0	1	1	1	0	1	1	0	0	1	0	1

Paso 2: Conversión a forma sistemática $[I_5 | P]$

Aplicamos operaciones de fila en GF(2) para obtener la identidad en las primeras 5 columnas:

Operaciones (ejemplo simplificado):

F_5 : sumar F_1

F_4 : sumar $F_1 + F_2$

F_3 : sumar F_2

F_2 : sumar F_1

Etc.

Matriz $G_{\text{sistemática}}$:

1	0	0	0	0		1	1	1	0	1	1	0	0	1	0
0	1	0	0	0		0	1	1	1	0	1	1	0	0	1
0	0	1	0	0		1	1	0	1	0	1	1	1	1	0
0	0	0	1	0		0	1	1	0	1	0	1	1	1	1
0	0	0	0	1		1	1	0	1	1	0	0	1	0	1

$$I_5 \quad P (5 \times 10)$$

Paso 3: Matriz de chequeo $H = [P^T | I_{10}]$

Transponemos P y agregamos la identidad 10×10 :

$P^T (10 \times 5)$:

1	0	1	0	1
1	1	1	1	1
1	1	0	1	0
0	1	1	0	1
1	0	0	1	1
1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
1	0	1	1	0
0	1	0	1	1

Matriz H completa:

1	0	1	0	1		1	0	0	0	0	0	0	0	0	0
1	1	1	1	1		0	1	0	0	0	0	0	0	0	0
1	1	0	1	0		0	0	1	0	0	0	0	0	0	0
0	1	1	0	1		0	0	0	1	0	0	0	0	0	0
1	0	0	1	1		0	0	0	0	1	0	0	0	0	0
1	1	1	0	0		0	0	0	0	0	1	0	0	0	0
0	1	1	1	0		0	0	0	0	0	0	1	0	0	0
0	0	1	1	1		0	0	0	0	0	0	0	1	0	0
1	0	1	1	0		0	0	0	0	0	0	0	0	1	0
0	1	0	1	1		0	0	0	0	0	0	0	0	0	1

- d. ¿Es $v(x) = 1 + x + x^5 + x^8 + x^{12} + x$ un polinomio código? Si no lo es, encuentre el síndrome.

Verificación de $v(x) = 1 + x + x^5 + x^8 + x^{12} + x^{14}$

División de $v(x)$ por $g(x)$

Dividimos: $(x^{14} + x^{12} + x^8 + x^5 + x + 1) \div (x^{10} + x^8 + x^5 + x^4 + x^2 + x + 1)$
 $x^4 + x^2 + 1$

$$x^{10} + x^8 + x^5 + x^4 + x^2 + x + 1 \mid x^{14} + x^{12} + x^8 + x^5 + x + 1$$

Paso 1: $x^{14} \div x^{10} = x^4$

$$x^4 \cdot (x^{10} + x^8 + x^5 + x^4 + x^2 + x + 1) = x^{14} + x^{12} + x^9 + x^8 + x^6 + x^5 + x^4$$

$$\begin{array}{r} x^{14} + x^{12} + x^8 + x^5 + x + 1 \\ - (x^{14} + x^{12} + x^9 + x^8 + x^6 + x^5 + x^4) \\ \hline \end{array}$$

$$x^9 + x^6 + x^4 + x + 1$$

Paso 2: $x^9 < x^{10}$, no se puede dividir más

$$\text{Resto: } r(x) = x^9 + x^6 + x^4 + x + 1$$

Conclusión

Como $r(x) \neq 0$, $v(x)$ NO es un polinomio código

Síndrome

$$S(x) = r(x) = 1 + x + x^4 + x^6 + x^9$$

$$\text{Vector síndrome: } S = (1, 1, 0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0)$$

e. Cuál es la capacidad de detección y corrección del código.

Cálculo de la distancia mínima:

Para un código lineal, $d_{\min}$ = peso mínimo de Hamming de las palabras código no nulas.

Del polinomio generador $g(x) = 1 + x + x^2 + x^4 + x^5 + x^8 + x^{10}$:

Peso de $g(x)$ = número de coeficientes = 1 (términos con coeficiente 1)

Contando: $1, x, x^2, x^4, x^5, x^8, x^{10} \rightarrow 7$ términos

Por tanto, $\text{peso}(g(x)) = 7$

Para códigos cíclicos: $d_{\min} \geq \text{peso}(g(x)) = 7$

Como $g(x)$ es una palabra código (mensaje $m(x)=1$), entonces $d_{\min} \leq 7$.

Capacidades:

Detección de errores:

- Detecta hasta $(d_{\min} - 1) = 6$ errores garantizado

- Corrige hasta $t = \frac{d_{\min}-1}{2} = \frac{6}{2} = 3$ errores

- Detecta ráfagas de longitud $\leq \text{grado}(g(x)) = 10$ bits